

Array

Gli array sono utilizzati per memorizzare più valori omogenei in una sola variabile. Si dichiarano utilizzando le parentesi quadre [].

Sintassi:

```
tipo[] nomeArray;
```

Possiamo riempire l'array direttamente in fase di dichiarazione, ponendo gli elementi tra parentesi graffe { }, separati da una virgola.

Esempio:

```
String[] auto = {"Volvo", "BMW", "Ford"};
```

In alternativa, è possibile utilizzare la parola chiave new per creare un array specificandone la dimensione tra parentesi quadre [].

Esempio:

```
String[] auto = new String[3];
```

Tipicamente, quando sappiamo già quali saranno i valori che popoleranno il nostro array utilizziamo la prima modalità, mentre quando i valori non sono noti utilizziamo la seconda, per creare un array vuoto della dimensione desiderata.

N.B.: la dimensione di un array è fissa, ovvero non è possibile modificarla in seguito alla sua creazione. Nel nostro esempio, l'array auto avrà sempre dimensione pari a 3.

Accedere ad un elemento

Per poter accedere ad un elemento e per modificarlo, dobbiamo specificare tra parentesi quadre l'indice dell'elemento stesso.

Esempio:

```
String[] auto = {"Volvo", "BMW", "Ford"};

System.out.println(auto[0]); //output: Volvo

auto[0] = "Citroen";
System.out.println(auto[0]); //output: Citroen
```

Lunghezza di un array

Possiamo sapere quanti elementi contiene un array (la sua dimensione) utilizzando la sua proprietà *length*.

Esempio:

```
String[] auto = {"Volvo", "BMW", "Ford"};

System.out.println(auto.length); //output: 3
```

Ciclare un array

È possibile scorrere tutti gli elementi di un array attraverso un ciclo for, sfruttando la proprietà *length* dell'array per specificare il numero di iterazioni da compiere.

Esempio:

```
String[] auto = {"Volvo", "BMW", "Ford"};

for(int i = 0; i < auto.length; i++)
    System.out.println(auto[i]);
```

In alternativa, possiamo anche utilizzare un for-each.

Esempio:

```
String[] auto = {"Volvo", "BMW", "Ford"};

for(String a : auto)
    System.out.println(a);
```

Array multidimensionali

Un array multidimensionale è un array che contiene al suo interno altri array. Tipicamente, si utilizzano array bidimensionali, in modo da organizzare i dati in una tabella/matrice.

Per creare un array bidimensionale andiamo a definire ogni riga all'interno di una coppia di parentesi graffe { }.

Per accedere ad un elemento dell'array bidimensionale, occorrerà specificare due indici: il primo è l'indice della **riga**, ovvero si indica quale tra gli array contenuti in quello principale si vuole andare a considerare, il secondo è l'indice della **colonna**, ovvero la posizione dell'elemento all'interno della riga selezionata.

Esempio:

```
int[][] numeri = {{1, 2, 3}, {4, 5, 6}};

System.out.println(numeri[0][1]); //output: 2
```

Attraverso i due indici è possibile non solo accedere, ma anche modificare un elemento, esattamente come in un array tradizionale.

La proprietà `length` ora assume un duplice significato in base a come viene utilizzata:

- se viene usata come proprietà dell'array "contenitore" ci viene restituito il numero di righe, ovvero quanti array sono contenuti in quello principale;
- se viene usata come proprietà della riga, ci restituisce il numero di colonne, ovvero quanti elementi contiene la riga stessa.

N.B.: è perfettamente lecito avere righe con lunghezze diverse (con un numero di elementi diverso).

Esempio:

```
int[][] numeri = {{1, 2, 3}, {4, 5, 6, 7, 8}};

System.out.println(numeri.length); //output: 2
System.out.println(numeri[0].length); //output: 3
System.out.println(numeri[1].length); //output: 5
```

Come per gli array tradizionali, possiamo usare la parola chiave `new` per creare un array bidimensionale vuoto con un certo numero di righe e colonne. È possibile creare un array bidimensionale specificando solo il numero delle righe e non quello delle colonne: in questo caso, ogni riga può essere inizializzata in maniera indipendente dalle altre ed avere, quindi, dimensione diversa. Tuttavia, non è possibile fare il contrario!

Esempio:

```
int[][] numeri = new int[3][3]; //array bidimensionale 3x3
int[][] numeri2 = new int[3][]; //3 righe e colonne non specificate
//ogni riga può avere dimensione diversa
numeri2[0] = new int[1];
numeri2[1] = new int[3];
numeri2[2] = new int[2];
int[][] numeri3 = new int[][3]; //errore!
```

Ciclare un array bidimensionale

Come vale per gli array tradizionali, anche uno bidimensionale può essere ciclato. Per farlo, possiamo utilizzare due cicli for (uno per le righe ed uno per le colonne) in combinazione con la proprietà length, oppure due cicli for-each.

Esempio:

```
int[][] numeri = {{1, 2, 3}, {4, 5, 6}};

for(int i = 0; i < numeri.length; i++){
    for(int j = 0; j < numeri[i].length; j++){
        System.out.println(numeri[i][j]);
    }
}

for(int[] riga : numeri){
    for(int n : riga){
        System.out.println(n);
    }
}
```